VG

Vyoma Gajjar

ENTERPRISE AI | ARCHITECTURE | ADOPTION

www.linkedin.com/in/vyomagajjar | mvgminfo@gmail.com

Map AI Usage to Deliverables.

Why your AI dashboard is incomplete. And how to build the engine that tells the truth.

LIGHTNING LESSON

May 13, 2026 · 11:30 AM PT · 30 minutes

28-PERSON ENGINEERING TEAM · ONE MONTH OF DATA

Five numbers describe the same team.

All from the same data. The vendor and the engine disagree by 61 points.

89%	29%	Δ 60.7	$16,800	16.9×
VENDOR DASHBOARD	ENGINE	THE GAP	ANNUAL SPEND	ROI MULTIPLE
25 of 28 seats had ≥ 1 prompt	8 of 28 shipped AI-traced work	percentage points · 17 in between	28 seats × $50/mo × 12 months	$283,200 value / $16,800 spend

HOW THE 16.9× ROI IS BUILT

```
value = 118 AI-traced artifacts/mo × $200 × 12 = $283,200/year
spend = 28 seats × $50/seat/month × 12 = $16,800/year
ROI = $283,200 / $16,800 = 16.9×
```

For every $1 spent on AI seats, $16.90 returns as engineering time. Replace the constants with your finance team's numbers.

THE PROBLEM

Vendor dashboards count logins.
Your CFO needs to count deliveries.

Every AI vendor ships an adoption dashboard that answers exactly one question: did this seat issue ≥ 1 prompt this month? Engineers who paste a prompt once and never use the answer count as 'adopted.' Engineers who ship AI-traced PRs every day count as 'adopted.' Same bucket. Same 89%. The number reassures the CFO. It does not predict anything.

WHAT VENDORS REPORT

Active seats. Total prompts. Login frequency. All of it correlates with 'tool is installed' : none of it correlates with 'output ships'.

WHAT THIS ENGINE REPORTS

AI-traced artifacts per person. Lineage ratio. Six archetypes. Per-OKR attribution. Recoverable dollars. The numbers your board will quote back to you.

THE PROMISE FOR THE NEXT 25 MINUTES

Three things you walk out with.

01

A working engine

~760 lines of Python you can read in one sitting. Standard library only. Every threshold visible. Replaces every vendor adoption tile you have.

02

The right question

Lineage. Not seats, not prompts, not telemetry. The fraction of shipped output that traces back to AI is the only metric that can't be gamed without doing real work.

03

A board narrative

Vendor says X. Engine says Y. Gap is dollars. Here is the trajectory. Here is the action list. Five sentences and a chart. That's the slide you brief from.

THE FRAMEWORK

Every engineer falls into one of six buckets.

Lineage decides. The other signals describe.

Power Adopter

High usage. High lineage. Ships AI-traced work.

Promote · pair with beginners

Quiet Producer

Low usage. High lineage. Uses Skills well.

Study · scale the pattern

Theater Performer

High usage. Zero lineage. Performs adoption.

Revoke seat · save $50/mo

Bot

Templated prompts. No lineage. Automation.

Revoke seat · investigate

Stalled Beginner

Low usage. Low lineage. Needs onboarding.

Pair · weekly check-in

Skeptic / Holdout

Zero usage. Watching from the sideline.

Re-pitch with proof or release

THE ARCHITECTURAL RULE

Lineage is the anchor.

Quantitative signals describe.

Lineage decides.

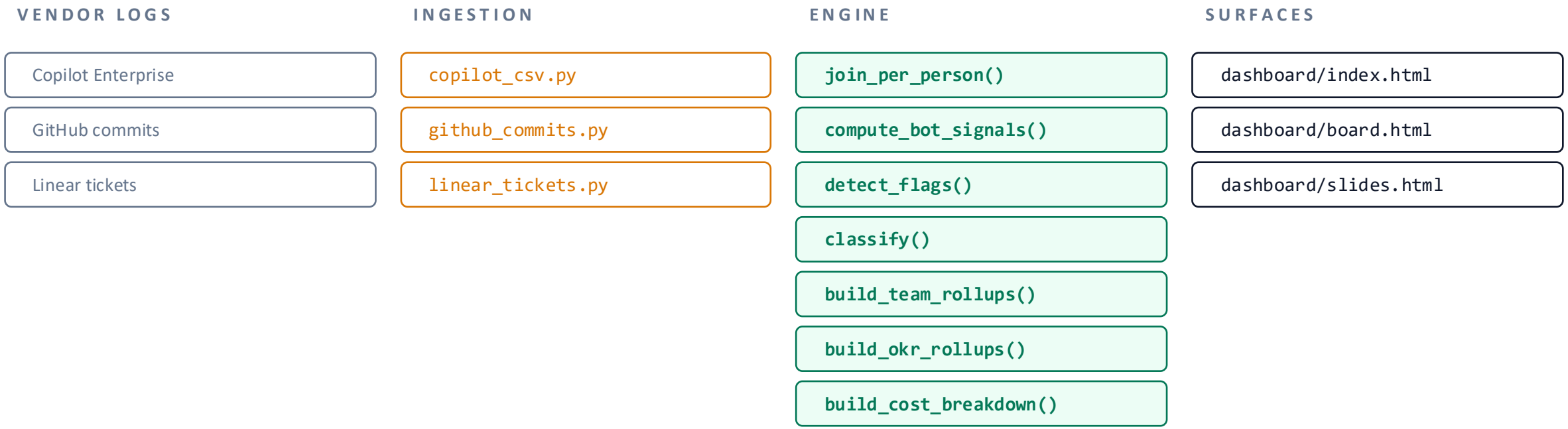
WHY THIS MATTERS

A senior engineer who runs five agents at night, uses templated /code-review prompts, and ships AI-traceable work would trip cadence (off-hours), entropy (templates), and bot scores under naive rules. The lineage anchor rules them out before any of those signals fire.

ARCHITECTURE · THE STACK

Three files. One JSON. Zero SaaS.

Every box on the diagram is a real file in the repo. About 1,800 lines of Python including the three vendor adapters.



Python standard library only. No pip install. No SaaS. React + Tailwind via CDN. The test gate is a single `python3 test_engine.py` command that returns PASS or FAIL.

Now we walk the code.

About 760 lines of Python. Standard library only. Every threshold visible. Nothing hidden in config. Five files, ten minutes.

01
data model
`UsageEvent · DeliveryEvent · OKR`

02
the join
`join_per_person()`

03
4 bot signals
`entropy · lineage · cadence · burst`

04
classifier
`6 rules, lineage anchored`

05
cost + ROI
`build_cost_breakdown()`

CODE · DATA MODEL

Three dataclasses define the world.

Usage from the AI vendor. Deliveries from version control. OKRs from leadership.

```python
@dataclass(frozen=True)
class UsageEvent:
    user_id: str
    prompt_template: str        # canonicalized, e.g. /code-review
    timestamp_utc: datetime

@dataclass(frozen=True)
class DeliveryEvent:
    user_id: str
    artifact_id: str          # commit SHA or PR id
    ai_traced: bool            # AI fingerprint detected in trailer
    okr_id: Optional[str]    # parsed from commit subject
    timestamp_utc: datetime
```

Note: ai_traced is a boolean, not a probability. We either find a fingerprint or we don't.

CODE · THE JOIN

Per-person summary in one function.

The math vendor dashboards never expose: lineage = AI-traced / total.

```
def join_per_person(usage, delivery):
    by_user = defaultdict(list)
    for d in delivery:
        by_user[d.user_id].append(d)

    rows = []
    for user_id in (all users):
        deliveries = by_user[user_id]
        traced     = sum(d.ai_traced for d in deliveries)
        rows.append(PersonSummary(
            user_id      = user_id,
            usage_count  = len([u for u in usage if u.user_id == user_id]),
            delivery_count = len(deliveries),
            ai_traced_count = traced,
            lineage_ratio = traced / max(1, len(deliveries)),
        ))
    return rows
```

THE METRIC

lineage_ratio

Fraction of this person's shipped artifacts that trace to AI. Zero means: theater. One means: every PR has an AI fingerprint. The number your CFO wants.

THE TRAP TO AVOID

max(1, n)

People with zero deliveries get lineage = 0/1 = 0, not divide-by-zero. Critical for new hires.

CODE · BOT SIGNALS

Four pure functions. Each independently testable.

Score 0 = clean human. Score 1 = bot-like. Lineage anchor overrides if any of these light up.

prompt_entropy

Shannon entropy on prompt templates. Three commands repeated = collapse.

no_lineage

How much shipped work has NO AI fingerprint. High means: usage doesn't reach work.

cadence_anomaly

Modal-band detection. If 80% of activity sits in one 12-hour window → clean schedule.

burst_pattern

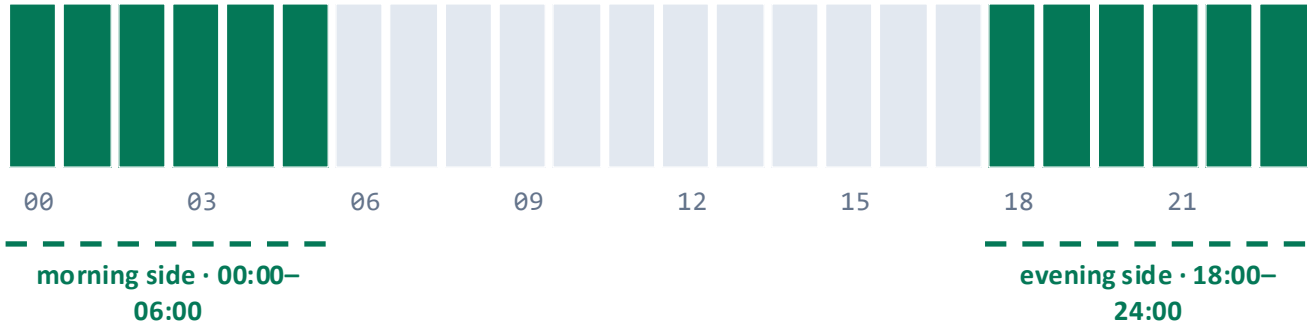
Unusual prompt-burst before review windows. Catches review-prep theater.

max() wins. If any one signal exceeds 0.7, the person enters bot review. Lineage above the floor cancels the verdict.

CODE · MODAL-BAND DETECTION

How a night-owl operator avoids the bot label.

VYOMA.GAJJAR · 190 PROMPTS · ACTIVITY BY HOUR (UTC)



Modal band: 18:00–06:00 · concentration = 100% · cadence signal = **0.00**

```python
def modal_band(timestamps):
    hours = [t.hour for t in timestamps]
    best_count, best_start = 0, 0
    for start in range(24):
        window = {(start + i) % 24 for i in range(12)}
        count = sum(1 for h in hours if h in window)
        if count > best_count:
            best_count, best_start = count, start
    return best_count / len(hours), best_start
```

If concentration ≥ 80%, cadence scores 0. She has a schedule. It's just not yours.

CODE · AI FINGERPRINT

What counts as AI-traced?

Six regex patterns over commit trailers. Any match → ai_traced = True.

```
AI_SIGNATURES = [
    r"Co-authored-by:.*Cursor",                    # Cursor co-author trailer
    r"Generated with .{0,4}Claude Code",           # Claude Code marker ( etc.)
    r"\bCo-authored-by: GitHub Copilot\b",         # Copilot Workspace
    r"Anthropic-Skill-Used:",                      # Skill invocation receipt
    r"\[ai-assisted\]",                            # team-internal tag
    r"AI-Tool: .+",                                # generic vendor trailer
]

def is_ai_traced(commit_body: str) -> bool:
    return any(re.search(pat, commit_body) for pat in AI_SIGNATURES)
```

Replace this list with your team's actual conventions. The shape of the function does not change.

CODE · OKR ATTRIBUTION

One regex turns commits into OKR coverage.

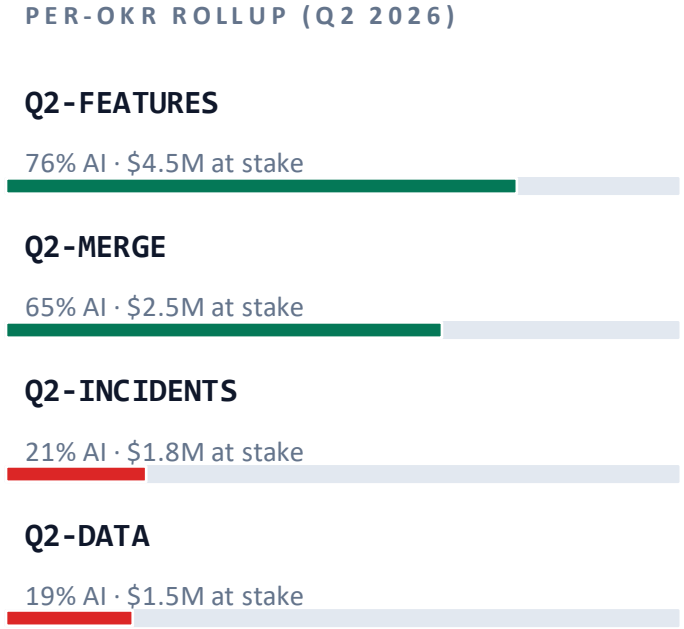
Subject lines like [Q2-FEATURES] auth flow → fold this commit into Q2-FEATURES.

```python
OKR_RE = re.compile(r"[\[\(](Q\d-[A-Z]+)[\]\)]")

def okr_id_from_subject(subject: str):
    m = OKR_RE.search(subject)
    return m.group(1) if m else None

# usage:
okr_id_from_subject("[Q2-FEATURES] add saml fallback")
→ "Q2-FEATURES"

okr_id_from_subject("fix typo in readme")
→ None
```



CODE · CLASSIFIER

Six rules. Order matters. Lineage anchored.

Read top to bottom. The Bot rule comes first because automation has no human shape.

```python
def classify(p, sig, flags):
    # Bot first. Automation has no human shape.
    if sig.entropy_collapse >= 0.7 and sig.no_lineage >= 0.7:
        return "Bot"
    # Edge cases: thin data → conservative bucket.
    if "inactive" in flags: return "Skeptic / Holdout"
    if "short-tenure" in flags: return _short_tenure(p)
    # Theater Performer: usage but no lineage.
    if p.usage_count >= 40 and p.lineage_ratio < 0.10:
        return "Theater Performer"
    # Power Adopter: usage AND delivery AND lineage.
    if p.usage_count >= 40 and p.lineage_ratio >= 0.20 \
            and p.delivery_count >= 25:
        return "Power Adopter"
    if p.usage_count == 0: return "Skeptic / Holdout"
    if p.usage_count <= 10 and p.delivery_count >= 25:
        return "Quiet Producer"
    return "Stalled Beginner"  # catchall
```

CODE · COST + ROI

A board-defensible dollar model in five lines.

```python
def build_cost_breakdown(reports) -> CostBreakdown:
    seats_total     = len(reports)
    annual_spend    = seats_total * 50 * 12            # $50/seat/mo enterprise tier
    waste_seats     = sum(1 for r in reports
                          if r.archetype in ("Theater Performer", "Bot"))
    annual_waste    = waste_seats * 50 * 12
    ai_traced       = sum(round(r.lineage_ratio * r.delivery_count) for r in reports)
    annual_value    = ai_traced * 200 * 12             # $200 saved per AI-traced artifact
    return CostBreakdown(spend=annual_spend, waste=annual_waste,
                         value=annual_value, roi=annual_value/annual_spend)
```

ANNUAL SPEND	ANNUAL WASTE	VALUE CREATED	ROI MULTIPLE
$16,800	**$3,600**	**$283,200**	**16.9×**
28 × $50 × 12	21% of spend (theater + bot)	118 AI-traced/mo × $200 × 12	value / spend

VERIFICATION

Tests pass on the synthetic team.

PASS = green-light demo. One command. Ten assertions. Pinned thresholds.

```
$ python3 test_engine.py

  vendor_dashboard_rate  = 89.3%
  real_adoption_rate     = 28.6%
  gap                    = 60.7 percentage points

  annual_AI_spend        = $16,800
  annual_waste           = $3,600   (21%)
  annual_value_created   = $283,200
  ROI multiple           = 16.9x

  teams:
    Platform   n=7  ai-traced= 58  $0/mo waste
    Core API   n=7  ai-traced= 26  $150/mo waste
    Frontend   n=5  ai-traced= 15  $100/mo waste
    Data       n=4  ai-traced= 19  $0/mo waste

PASS  (all checks green)
```

LIVE DEMO · DASHBOARD WALKTHROUGH

From numbers to action plan.

Three panels you operate from. The fourth answers: 'is the gap closing?'

ARCHETYPE MIX

Stacked bar per team. The eye lands on Frontend: 4 power adopters, 3 stalled.

OKR HEATMAP

Per-OKR AI attribution. Q2-FEATURES is red. That's your QBR conversation.

ACTION PLAN

Revoke 6 accounts: $3,600/yr recovered. Promote 5 patterns: pair with stalled.

12-WEEK TREND

Org attribution 18% → 48%. Intervention at week 6 worked.

Three sentences your CFO can repeat.

01 The vendor says 89% of the org adopted AI. The engine says 29% shipped AI-traced work. The 60-point gap is $3,600 of recoverable seats and $283,200 of unrealized value.

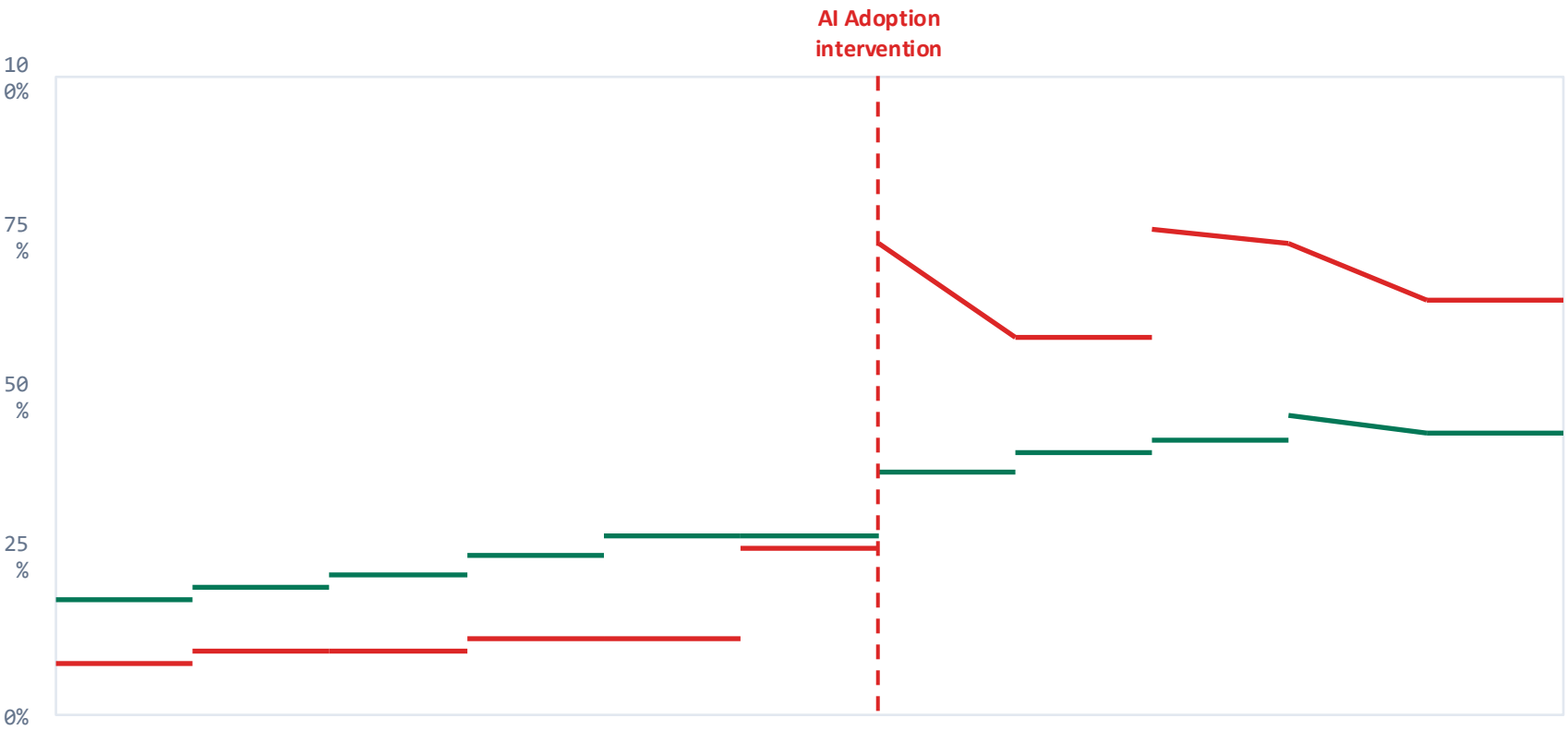
02 Frontend has eight percent AI attribution on a $4.5M OKR. That is the team your VP needs to brief next quarter, not the one with the most prompts.

03 We revoke six accounts, promote five power-adopter patterns, and retest in twelve weeks. Trajectory is published. If the line bends, the framework works.

12-WEEK TRAJECTORY · THE VERIFICATION

Is the gap closing?

Org-wide AI attribution 18% → 48%. Q2-FEATURES 8% → 76%. Intervention week 6.



18% → 48%

ORG-WIDE AI ATTRIBUTION

8% → 76%

Q2-FEATURES (the focus OKR)

Annual value: **$283k → $742k** · ROI: **16.9× → 44.1×** *over the same 12 weeks. The line that bends most matters most.*

Why your senior engineer isn't a bot.

An engineer runs five agents between 6 pm and 6 am. Templated prompts. Off-hours. Naive rules flag them. Our engine does not.

190
PROMPTS / MO

45
ARTIFACTS SHIPPED

53%
AI-TRACED LINEAGE

ARCHETYPE

Power Adopter

night-owl | multi-agent-operator

NAIVE RULE

Off-hours activity + templated prompts → flagged as Bot. She loses her seat. The framework loses the room.

LINEAGE ANCHOR

53% of her output traces to AI. Lineage above the floor cannot be Bot, regardless of cadence or entropy. **Trust earned.**

The numbers you take home.

Five facts every CTO in the room will quote back next Monday.

21%

of typical seats produce zero AI-traceable output

4×

Theater Performers generate roughly four times more prompts than Power Adopters

60+ pts

Gap between vendor-reported adoption and real adoption

3 / 28

are not human. Every enterprise has had a 'paid AI seat' for a literal bot

12 lines

of Python classify the entire org. The framework is a decision table, not a model

Lineage ratio is the only AI adoption metric that cannot be gamed without doing real work. *Vendors will not surface it. You have to build it.*

DEPLOYMENT · YOUR NEXT MONDAY

Four weeks. One engineer. Zero new vendors.

WEEK 1	**Export, ingest, classify**	Export Copilot CSV. Pull git log. Run engine. Get archetype mix for one team.
WEEK 2	**Tune thresholds, validate**	Walk results with two senior engineers. Adjust constants for your seat cost and stack. Tests stay green.
WEEK 3	**Wire OKR ids**	Add [Qx-OKR] tags to commit subjects. Backfill last sprint. OKR heatmap goes live.
WEEK 4	**Publish board view**	Print-to-PDF the QBR slide. Forward to your VP. Schedule a 30-min review with engineering leadership.

Stop measuring logins.
Start measuring lineage.

Thank you.

Vyoma Gajjar
ENTERPRISE AI | ARCHITECTURE | ADOPTION

www.linkedin.com/in/vyomagajjar
mvgminfo@gmail.com